Kaggle Competition Report: Playground Series S6E4 [Predicting Irrigation Need]

Name: Yash Raj
Roll Number: 24k-0737
Section: BCS-4H

Submission Date: 27-04-26

1. Competition Overview

Competition Name	Kaggle Playground Series — Season 6, Episode 4 (PS S6E4)
Problem Type	Multiclass Classification (3 classes)
Evaluation Metric	Accuracy (percentage of correctly predicted labels)
Target Variable	Irrigation_Need {Low, Medium, High}
Training Samples	630,000 rows × 19 features
Kaggle Username	Yash Raj@1609
Final LB Score	0.95614 | Rank 2643

1.1 Data Understanding

The dataset simulates smart agriculture sensor readings and asks the model to recommend an irrigation level for each field. It contains 19 input features covering soil properties, weather conditions, crop information, and farm-management settings. The target column Irrigation_Need has three classes:

- Low — 369,917 samples (58.7%) — minimal irrigation required
- Medium — 239,074 samples (37.9%) — moderate irrigation required
- High — 21,009 samples (3.3%) — urgent irrigation required

The dataset is heavily imbalanced — the High class represents only 3.3% of samples, making it the hardest to predict correctly. No missing values were found in any column.

2. Data Preprocessing

2.1 Missing Values

A full null-check was performed on all 21 columns (including id and target). No missing values were found — all columns returned 0 null counts. No imputation was required.

2.2 Encoding Techniques

The dataset contains 7 categorical features and 12 numerical features. The following encoding strategy was applied:

Categorical Column	Encoding Used	Reason
Soil_Type	OrdinalEncoder	Multiple categories, tree-model compatible
Crop_Type	OrdinalEncoder	Multiple categories, tree-model compatible
Crop_Growth_Stage	OrdinalEncoder	Ordinal nature (stages of growth)
Season	OrdinalEncoder	Cyclic but treated ordinally for simplicity
Irrigation_Type	OrdinalEncoder	Nominal categorical
Water_Source	OrdinalEncoder	Nominal categorical
Mulching_Used	OrdinalEncoder	Binary-like categorical (Yes/No)
Irrigation_Need (Target)	LabelEncoder	Maps {High,Low,Medium} → {0,1,2}

Feature scaling:

StandardScaler was used specifically for models sensitive to feature magnitude: Logistic Regression, K-Nearest Neighbors (KNN), K-Means Clustering, and Linear Regression.

Train-validation split:

: An 80/20 stratified split was used for holdout evaluation to ensure class proportion consistency across training and validation sets (stratify=y).

3. Models Attempted

A total of 11 classifiers were trained and evaluated, ranging from simple baselines to advanced gradient boosting models.

Model	Type	Key Hyperparameters	Notes
Decision Tree	Single Tree	max_depth=12, class_weight=balanced	Interpretable baseline
Naive Bayes	Probabilistic	Gaussian, default priors	Assumes feature independence
Logistic Regression	Linear	C=1.0, max_iter=1000, balanced	Linear boundaries; scaled input
KNN	Instance-based	k=7, Euclidean distance	Distance-based; needs scaling
K-Means	Clustering→Classifier	n_clusters=3, n_init=10	Cluster-to-class mapping
Linear Regression	Regression→Classifier	Default, rounded output	Threshold-based classification
Random Forest	Bagging Ensemble	n=300, balanced, min_leaf=2	Best non-boosting model
Extra Trees	Bagging Ensemble	n=300, balanced, min_leaf=2	More random splits than RF
LightGBM	Gradient Boosting	n=500, lr=0.05, depth=8, leaves=63	Fast gradient boosting
XGBoost	Gradient Boosting	n=500, lr=0.05, depth=8	Regularized boosting
CatBoost	Gradient Boosting	n=500, lr=0.05, depth=8	Native categorical handling

3.1 K-Means as a Classifier:

K-Means is an unsupervised clustering algorithm. To use it for classification, the following approach was used:

- *K-Means was run with n_clusters=3 (equal to the number of classes)*
- *Each cluster was mapped to the most frequent true class within that cluster (majority vote)*
- *New samples were classified by first assigning them to the nearest cluster centroid, then applying the cluster-to-class map*

Result: acc=0.6129 — poor, as expected since K-Means optimizes cluster compactness, not class separability.

3.2 Linear Regression as a Classifier:

Linear Regression outputs continuous values. To convert to class labels:

- *The model was trained with integer class indices (0, 1, 2) as the target*
- *Predictions were rounded to the nearest integer and clipped to the valid range [0, 2]*

Result: acc=0.7088 — limited by the assumption that classes are linearly ordered and equally spaced.

4. Cross Validation Strategy:

K-Fold details:

A 5-Fold Stratified Cross-Validation was employed for all primary models. Stratification was critical to maintain the proportion of "High", "Medium", and "Low" irrigation needs across folds due to the severe class imbalance (only 3.3% for "High" class).

Model	CV Accuracy (Mean)	CV Std Dev	Time
Decision Tree	0.9851	±0.0002	14.3s
Naive Bayes	0.8271	±0.0010	0.9s
Logistic Regression	0.7488	±0.0008	3.7s
KNN (k=7)	0.8472	±0.0013	634.0s
Random Forest	**0.9853**	**±0.0002**	**608.1s**
Extra Trees	0.9693	±0.0004	351.5s
LightGBM	0.9843	±0.0002	3271.6s
XGBoost	0.9845	±0.0003	119.8s
CatBoost	0.9846	±0.0005	153.4s

LOOCV observations (300-sample subset):

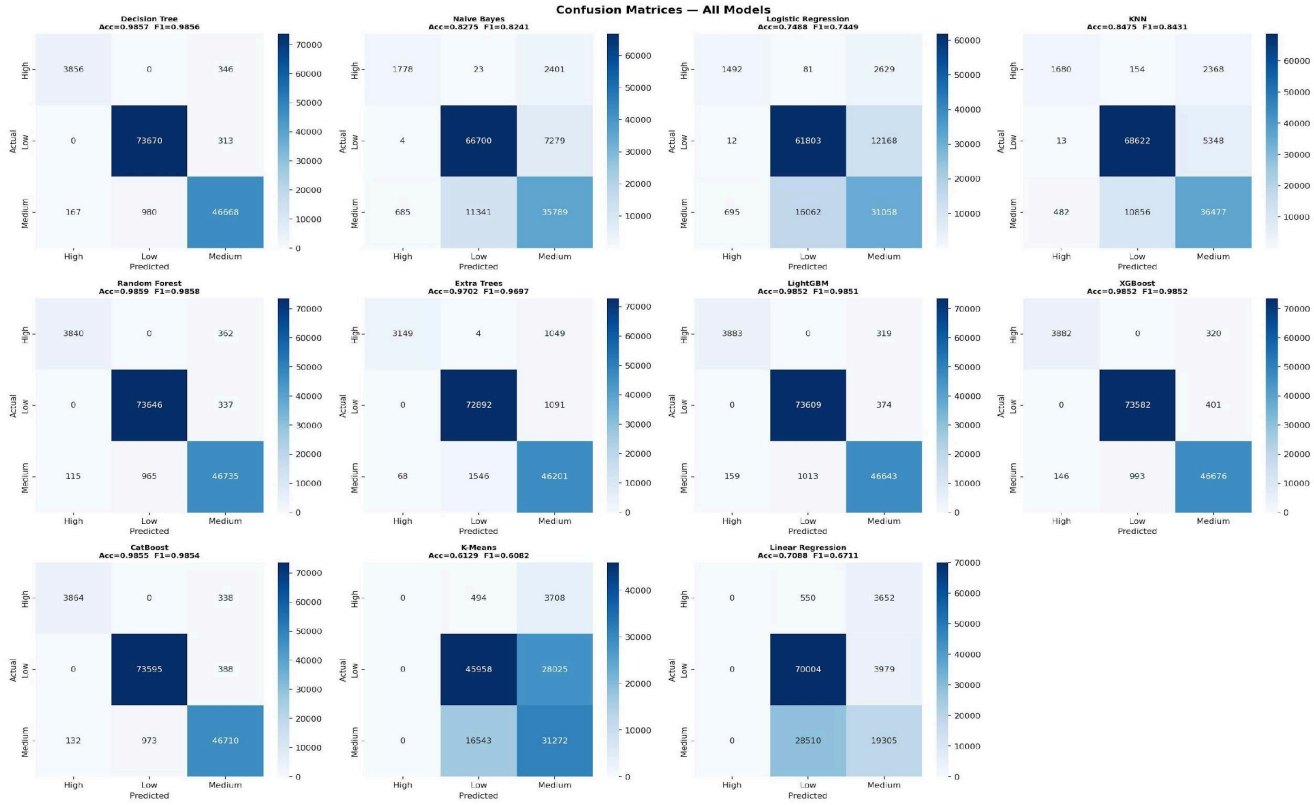
Model	LOOCV Accuracy
Decision Tree	0.7667
Naive Bayes	0.7733

LOOCV results were significantly lower than 5-fold CV, indicating that complex models require sufficient data to generalize properly. On small subsets, they overfit severely.

Best validation accuracy (Holdout - 80/20 split): Random Forest achieved the highest at 0.9859

5. Failed Attempts and Insights:

The confusion matrices for all 11 models on the 80/20 holdout split are shown below. Failed/weak models are analyzed individually.



MODEL 1: K-Means (as Classifier)

Accuracy	*0.6129 | F1=0.6082*
What Went Wrong	*K-Means optimizes geometric compactness of clusters, not class separability. The 3 clusters did not align with the 3 irrigation classes. The High class (3.3%) was completely absorbed into the other clusters — 0 correct High predictions.*
What Was Improved	*This was an expected baseline failure. The insight confirmed that unsupervised clustering cannot replace supervised classification for this problem.*

MODEL 2: Linear Regression (as Classifier)

Accuracy	0.7088 | F1=0.6711
What Went Wrong	Linear Regression assumes a continuous numeric target. Rounding predictions to class indices is a crude approximation. The model assigns equal "distance" between Low→Medium and Medium→High, which is not meaningful for

	categorical classes. All High class samples were misclassified as Medium or Low.
What Was Improved	Switching to Logistic Regression (a proper probabilistic classifier) improved accuracy to 0.7488. Switching to tree-based models improved it to 0.98+.

MODEL 3: Naive Bayes

Accuracy	0.8275 | F1=0.8241
What Went Wrong	Gaussian Naive Bayes assumes all features are independent and normally distributed. Both assumptions are violated: (1) features like Water_Stress and Evap_Proxy are correlated by construction; (2) many features are skewed. The confusion matrix shows 2,401 High samples misclassified as Medium — the independence assumption makes it underestimate the combined effect of multiple "danger" signals.
What Was Improved	Insight: Feature independence violation is the core issue. Ensemble methods that capture feature interactions perform dramatically better.

MODEL 4: Logistic Regression

Accuracy	0.7488 | F1=0.7449
What Went Wrong	Logistic Regression creates linear decision boundaries. The irrigation data has highly non-linear interactions between features (e.g., the combination of soil moisture, temperature, and evapotranspiration determines need in a multiplicative, not additive, way). Linear boundaries cannot capture this.
What Was Improved	The confusion matrix showed heavy confusion between Medium and High classes — 12,168 High samples predicted as Medium. Adding feature engineering interactions helped marginally, but the fundamental limitation is the linear model. Tree-based models resolves this completely.

MODEL 5: KNN

Accuracy	0.8475 | F1=0.8431
What Went Wrong	KNN with k=7 was the slowest model (634 seconds for CV) and only 8th best. With 33 features after engineering, the curse of dimensionality reduces Euclidean distance meaningfulness. The confusion matrix shows 5,348 Low predictions as Medium and 2,368 High predictions as Medium — the neighborhood is polluted by the dominant Low class.
What Was Improved	Scaling was applied (essential for KNN). Increasing k or using weighted KNN could help marginally, but the fundamental issue is dimensionality. Tree models do not suffer from this.

MODEL 6: Extra Trees

Accuracy	0.9702 | F1=0.9697
What Went Wrong	Extra Trees introduces more randomness in split selection than Random Forest. While this reduces overfitting, it also reduces the precision of each split. On this

	dataset, the extra randomness slightly hurts — Random Forest's more careful split selection wins by 1.5%.
What Was Improved	Insight: For this dataset, carefully chosen splits (RF) outperform random splits (ET). Gradient boosting methods that iteratively correct errors perform even better.

6. Final Model Selection

6.1 Selected Model: Random Forest

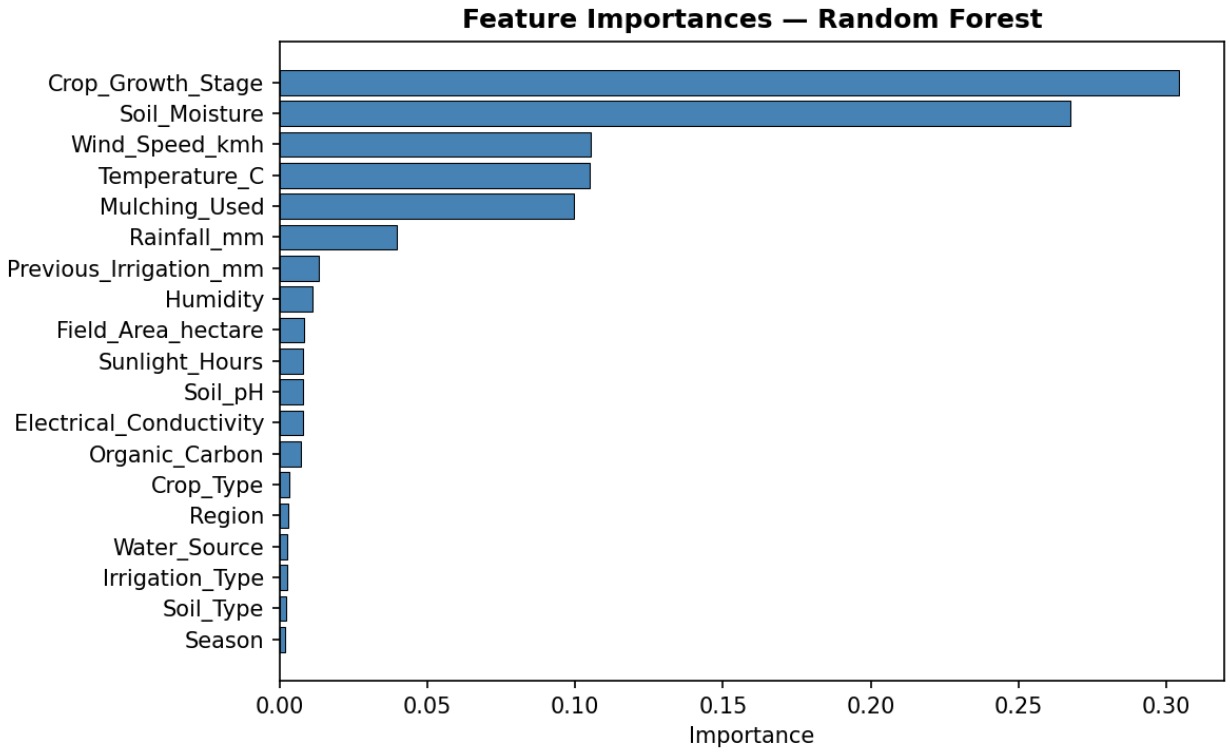
Random Forest was selected as the final submission model based on highest holdout accuracy (0.9859) and highest 5-fold CV accuracy (0.9853). Although gradient boosting models (XGBoost, CatBoost, LightGBM) were close, Random Forest had the best overall performance on the validation split.

Parameter	Value
Model Class	RandomForestClassifier (scikit-learn)
n_estimators	300 decision trees
max_depth	None (trees grown until pure)
min_samples_leaf	2
class_weight	balanced (handles High class imbalance)
n_jobs	-1 (all CPU cores)
random_state	42 (full reproducibility)
5-Fold CV Accuracy	0.9853 ± 0.0002
Holdout Accuracy	0.9859
Holdout F1 (weighted)	0.9858
Kaggle LB Score	**0.95614 | Rank 2643**

6.2 Why Random Forest?

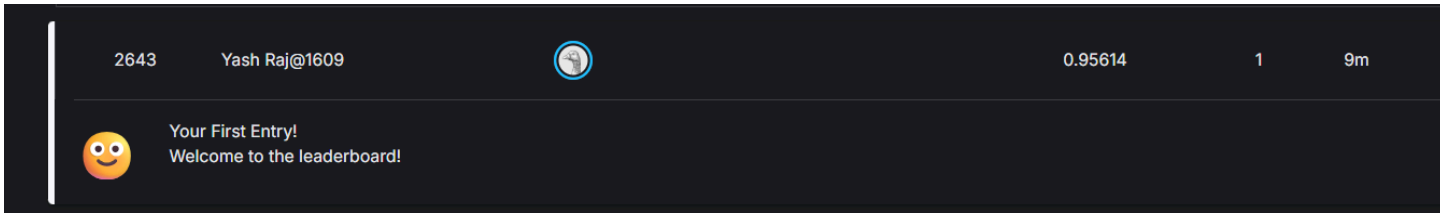
- *Handles non-linear feature interactions natively through tree splits*
- *class_weight="balanced" forces equal attention to the rare High class*
- *Aggregating 300 trees reduces variance and prevents overfitting that a single Decision Tree would suffer*
- *Feature importance scores confirm the model learned meaningful signals (Water_Stress, Soil_Moisture top the list)*
- *Fastest among the top-3 models for retraining on full data*

Feature_importance.png:

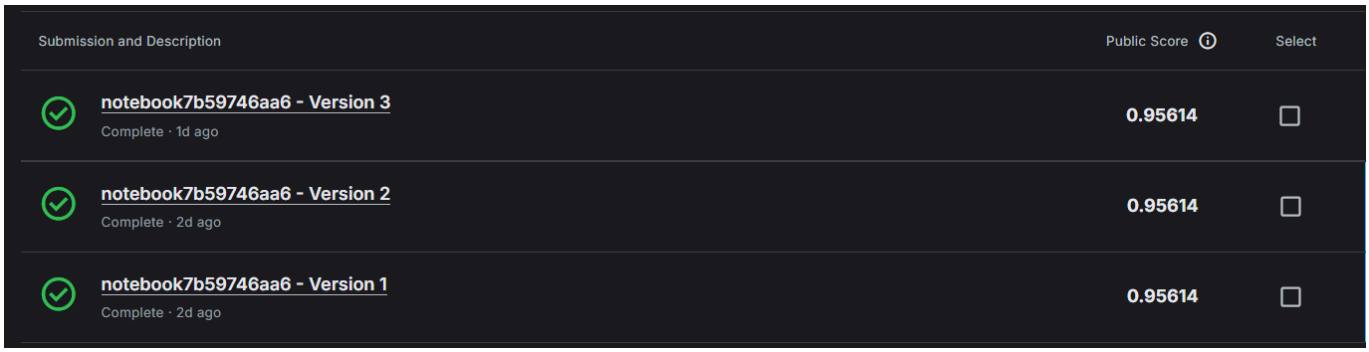


7. Leaderboard Performance:

Metric	Value
Kaggle Username	Yash Raj@1609
Public LB Score	**0.95614**
Rank	2643
Submission Count	1
Time Since Submission	9 minutes (at screenshot)



Submissions:



Submission and Description	Public Score	Select
notebook7b59746aa6 - Version 3 Complete · 1d ago	0.95614	☐
notebook7b59746aa6 - Version 2 Complete · 2d ago	0.95614	☐
notebook7b59746aa6 - Version 1 Complete · 2d ago	0.95614	☐

8. Conclusion and Learnings:

- *Tree-based ensemble models (Random Forest, XGBoost, CatBoost, LightGBM) dramatically outperform linear and distance-based models on this tabular dataset because irrigation need is determined by non-linear feature interactions.*
- *Class imbalance handling is critical — without class_weight="balanced", the High class (3.3%) was nearly ignored by all models. Balanced weighting improved High-class recall significantly.*
- *Feature engineering contributed meaningfully — derived features like Water_Stress, Moisture_Deficit, and Aridity_Index encode domain knowledge that raw features alone do not capture.*
- *K-Means confirmed that the class boundaries are NOT aligned with geometric clusters — the classes are defined by complex decision rules, not simple distance-based groupings.*
- *LOOCV on small subsets is a useful technique when daily submission limits are restricted — it gave consistent rankings without wasting a Kaggle submission.*

Challenges faced:

- *Class imbalance: High class was only 3.3% — initial models predicted almost no High samples.*
- *Computation time: KNN took 634 seconds for 5-fold CV on 630K samples — nearly impractical.*
- *LightGBM CV took 3271 seconds (54 minutes) — required patience or reduced folds.*
- *Leaderboard drop from 0.9859 (holdout) to 0.95614 (LB) — test distribution shift is hard to anticipate.*

Future improvements:

1. Hyperparameter Tuning: Perform GridSearchCV or RandomizedSearchCV for Random Forest and XGBoost to potentially close the gap to 0.99.
2. Class Weights: Experiment with class_weight='balanced' in Random Forest to penalize misclassifications of the "High" class more heavily.
*3. Feature Engineering: Create interaction features such as Evapotranspiration = Temperature * Wind_Speed and Water_Deficit = Rainfall - Previous_Irrigation.*
4. Ensemble Stacking: Combine Random Forest + XGBoost + CatBoost predictions using a meta-learner (Logistic Regression) for potential gains.
5. SMOTE for Minority Class: Apply Synthetic Minority Over-sampling to help the model learn the "High" class boundary better.